COURSE OUTCOMES: CO4, CO5, CO6 (LEVEL 2 & 3) — 30 MARKS EVALUATION

MediTrack: Smart Hospital Care System with MongoDB & AI Vector Search

Designing Document Schemas, Engine Validation ($jsonSchema), Atomic Array Telemetry, Multi-Stage Aggregation Pipelines, and Clinical Vector Retrieval

Domain: Healthcare Informatics & NoSQL	**Target DB:** MongoDB 6.0+ / Atlas	**Language:** Python 3.10+ (PyMongo)
Author: Final Year CSE Student	**Web Stack:** Flask + Tailwind + Chart.js	**Vector Model:** 64-Dim Dense Embeddings

1. The Problem Statement & Real-World Relevance

Why modern clinical environments demand document-oriented architectures over legacy RDBMS

The Relational Healthcare Bottleneck

- In SQL databases, patient charts are fragmented across 8+ normalized tables (Patients, Vitals, Prescriptions, Doctors, Billing).
- Retrieving an emergency patient chart requires 5+ table JOINs, causing high latency under concurrent ward loads.
- Diagnostic fields evolve constantly (e.g. ICU telemetry, MRI scans), requiring costly schema migrations.

The MongoDB Solution (MediTrack)

- **Zero-Join Lookups:** Co-locates demographics, continuous vitals, and medications in a single BSON document ($O(1)$ read).
- **Atomic Streaming:** Appends fresh vital telemetry using MongoDB's atomic `$push` without locking records.
- **Embedded AI Vector Search:** Stores 64-dim embeddings directly in the document for semantic symptom search.

2. Database Architecture: Hybrid Document Design

Disciplined engineering balance between high-locality embedding and scalable referencing

EMBEDDED DOCUMENTS (1-to-Few)

Fields: `vitals_history`, `prescriptions`, `billing`

- **High Read Locality:** Queried together with the patient in 99% of requests.
- **Bounded Size:** Average 20–50 readings per stay (<10 KB total, far below 16MB BSON limit).
- **Atomic Mutation:** Allows single-operation updates via `$push` and `$inc`.

REFERENCED COLLECTIONS (1-to-Many)

Fields: `assigned_doctor_id` → `doctors` collection

- **Unbounded Growth:** A doctor treats thousands of patients over years.
- **Zero Duplication:** Doctor profiles, fees, and ratings exist in only one place.
- **Cross-Collection Analytics:** Seamlessly joined in analytics via `$lookup`.

3. Engine-Level Schema Validation ($jsonSchema)

Enforcing strict medical type safety, regex patterns, and range boundaries inside MongoDB

Validation Specification

- **patient_id:** Regex pattern `^PAT-[0-9]{4}$`
- **age:** BSON int, minimum 0, maximum 125
- **gender & blood_group:** Strict Enum taxonomies
- **vitals_history.heart_rate:** Int bounds 30–240 BPM
- **vitals_history.oxygen_level:** Int bounds 50–100 SpO2 %
- **Action:** `validationAction: 'error'` blocks writes.

Sample Rejection Test Output

Testing invalid document insertion:

- `patient_id: 'INVALID_ID'` (violates regex)
- `age: 155` (violates maximum 125)
- `gender: 'Alien'` (violates enum)

Result: WriteError: Document failed validation

Proves database engine guarantees clinical safety independent of client code.

4. PyMongo CRUD Operations & Array Operators

Demonstrating high-concurrency atomic mutations and simultaneous array condition matching

CREATE	READ ($elemMatch)	UPDATE ($push / $inc)	DELETE (Soft Delete)
Inserts validated document with auto-generated 64-dim vector embedding. `db.patients.insert_one(doc)`	Matches patients where heart_rate ≥ 100 AND SpO2 ≤ 95 on the **exact same reading**. `$elemMatch: {hr, spo2}`	Appends vitals to array via `$push`. Increments billing via `$inc` with zero read-modify-write race conditions.	HIPAA compliant logical archive. `$set: {status: 'Archived', is_archived: True}`

5. Aggregations: Operations & Staffing Intelligence

Extracting high-level executive insights without transferring raw collections over the network

PIPELINE 1: Department Financials & Volumes

Stages: `$match` → `$group` → `$project` → `$sort`

- Groups all admitted patients by `department`.
- Computes total active census, average age, and aggregate billing.
- **Result:** Orthopedics ($4,920 total, avg $2,460), Cardiology ($3,835 total).

PIPELINE 2: Doctor Workload ($lookup Join)

Stages: `$lookup` → `$project` → `$sort`

- Joins `doctors` with `patients` where `doctor_id == assigned_doctor_id`.
- Computes caseload size via `$size` and total billing contribution.
- **Result:** Dr. Sarah Jenkins (3 patients, $3,835 rev), Dr. Emily Taylor (2 patients, $4,920 rev).

6. Aggregations: Clinical Triage & Executive KPI

Demonstrating array deconstruction ($unwind) and single-query parallel dashboards ($facet)

PIPELINE 3: Critical Vitals Triage ($unwind)

- Flattens embedded `vitals_history` array into distinct document stream.
- Matches anomalous physiological thresholds (HR ≥ 100 BPM or SpO2 ≤ 95%).
- Ranks critical patient alerts across hospital by heart rate severity.

PIPELINE 4: Pharmacy Prescription Frequency

- Flattens `prescriptions` array and groups by drug name to optimize stock.

PIPELINE 5: Executive Multi-Faceted KPI ($facet)

Single Database Round-Trip: Executes multiple independent sub-pipelines concurrently on the same input documents:

- **Sub-Pipeline A:** Inpatient status breakdown (5 Admitted, 2 Observation, 1 Discharged).
- **Sub-Pipeline B:** Financial realization breakdown ($6,295 Paid, $3,940 Claimed, $2,095 Pending).
- **Advantage:** 66% reduction in network round-trips for executive dashboard generation.

7. MongoDB as a Vector Database & AI Clinical Assistant

Querying clinical cases by semantic meaning rather than exact keyword matches

Why MongoDB for Vectors?

- **Single Data Store:** Eliminates syncing between MongoDB and Pinecone/Weaviate (zero sync drift).
- **Unified Document:** Transactional vitals, medical notes, and 64-dim embeddings live together.
- **Atlas Vector Search:** Uses native HNSW index via `$vectorSearch` aggregation operator.
- **Hybrid Filtering:** Allows combining vector similarity with traditional metadata filters (e.g. department).

Live AI Clinical Assistant Workflow

1. Physician types: *'acute chest pain, tachycardia, and suspected blockage'*.
2. System generates a 64-dimensional query vector Q.
3. Performs cosine similarity search across MongoDB patient charts.
4. **Result:** Matches PAT-1001 (88.4%) and PAT-1005 (82.1%) with **High Confidence**.
5. AI synthesizes diagnostic summary and suggests common medications (Atorvastatin, Metoprolol).

8. Full-Stack Web Portal Architecture

Clean, responsive clinical dashboard powered by Flask, Tailwind CSS, and Chart.js

Route	View Name	Key Architectural Capabilities Demonstrated
`/`	Executive Dashboard	Real-time KPI metrics, department bed occupancy meters, and recent admissions.
`/patients`	Patient Directory	Filterable table with department dropdown, status badges, and dynamic regex search.
`/patient/<id>`	Patient Chart & Telemetry	Interactive Chart.js vital curves, prescriptions, and modals for atomic $push and $inc updates.
`/patients/new`	Admission Form	Interactive test form with 'Fill Invalid Sample' button demonstrating $jsonSchema rejection.
`/analytics`	Aggregation Hub	Visual charts and collapsible query code viewers for Pipelines 1 to 5.
`/ai-search`	AI Vector Search	Natural language clinical query box, similarity meters, and Atlas $vectorSearch syntax.

9. Conclusion & Deliverables Summary

Complete satisfaction of Course Outcomes CO4, CO5, and CO6 with enterprise deliverables

Submitted Project Deliverables

- **Full Codebase:** Complete Python/PyMongo full-stack repository.
- **Formal PDF Report:** `SmartHospital_Project_Report.pdf`
- **Presentation Deck PDF:** `SmartHospital_Presentation.pdf`
- **Raw MongoDB Script:** `scripts/mongo_queries.js` for mongosh / Compass.
- **Execution Proof Log:** `output_demonstration.txt` with test records.
- **Viva Defense Guide:** 15 model Q&A; with schema justifications.

Key Academic Takeaways

- Document databases excel at hierarchical, co-located healthcare telemetry.
- Hybrid data modeling prevents relational JOIN bottlenecks while avoiding data duplication.
- Database-level schema validation provides structural safety in schemaless systems.
- MongoDB functions as a high-performance unified Operational and Vector Database.

Thank you! Questions and Live Demonstration welcome.